

Universidad Técnica Federico Santa María

ICN292 · Sistemas de Información para la Gestión

Laboratorio 5
Consultas SQL para Andes Bike

Fernando Beiza

Noviembre 2026

Repositorio GitHub

https://github.com/FBeiza21/ICN292-Lab5-Beiza_Acu-a-Fernando

Índice

1. Resumen ejecutivo 3
2. Base de datos y metodología 3
3. Desarrollo de las consultas 4
3.1. Cantidad de ventas por vendedor 4
3.2. Ventas y unidades por tienda 5
3.3. Tiendas sin vendedores asignados 6
3.4. Modelos sin ventas en junio de 2025 7
3.5. Vendedores sin ventas y su tienda 8
3.6. Pares de vendedores de una misma tienda 9
3.7. Tiendas con más de $2.000.000 en Montaña y Ruta 10
3.8. Bicicletas con precio superior al promedio 11
3.9. Vendedores sobre el promedio de unidades por venta 12
3.10. Monto total vendido por vendedor 13
4. Análisis de resultados y conclusiones 14

1. Resumen ejecutivo

El Laboratorio 5 tuvo como objetivo aplicar consultas SQL a la base relacional de Andes Bike para responder preguntas sobre sus ventas, vendedores, sucursales y catálogo. La base reunió cuatro tiendas, ocho vendedores, ocho modelos y 18 registros de venta entre marzo y junio de 2025. En conjunto, las ventas representaron 35 bicicletas y un monto de $16.969.650.

Concepción lideró en unidades vendidas con 14, mientras que Providencia registró la mayor cantidad de ventas, con 7. La Serena fue la única tienda sin vendedores asignados. Cuatro modelos quedaron sin ventas durante junio de 2025 y el precio promedio del catálogo fue de $657.490. Las capturas de MySQL Workbench muestran los resultados utilizados para desarrollar cada consulta.

2. Base de datos y metodología

2.1. Estructura de la base

La base andesbike se organizó en cuatro tablas. Las relaciones permiten asociar una venta con su vendedor y su modelo de bicicleta. La sucursal se obtiene a partir de la asignación registrada para el vendedor.

Tabla	Registros	Información utilizada en las consultas
tiendas	4	Identificador, nombre y comuna de la sucursal.
vendedores	8	Identificador, nombre, tienda asignada y fecha de ingreso.
bicicletas	8	Identificador, modelo, categoría y precio del catálogo.
ventas	18	Identificador de venta, vendedor, bicicleta, fecha y unidades.

Tabla 1: Tablas de la base de datos y campos utilizados.

2.2. Criterios de cálculo y verificación

Se contó cada fila de ventas como una venta y se sumó su campo unidades para obtener el volumen vendido. El monto de cada registro se calculó multiplicando las unidades por el precio del modelo. Se contrastaron las salidas con la carga oficial y se revisó que los totales de ventas, unidades y monto coincidieran con los datos de origen.

3. Desarrollo de las consultas

3.1. Cantidad de ventas por vendedor

Se buscó determinar cuántas ventas registró cada vendedor de Andes Bike, incluyendo a quienes no tuvieron actividad en la base.

Procedimiento

La consulta parte de la tabla vendedores y la relaciona con ventas mediante un LEFT JOIN por id_vendedor. De esta forma, los ocho vendedores aparecen en el resultado aunque no exista una venta asociada. El agrupamiento por identificador y nombre genera una fila por persona.

La función COUNT(v.id_venta) cuenta solo los identificadores de venta que tienen valor. En una fila sin coincidencia, ese campo queda en NULL y el conteo devuelve cero. El resultado se ordena primero por cantidad descendente y luego por nombre para resolver los empates.

```
SELECT
   ve.nombre AS vendedor,
   COUNT(v.id_venta) AS cantidad_ventas
FROM vendedores AS ve
LEFT JOIN ventas AS v
   ON v.id_vendedor = ve.id_vendedor
GROUP BY
   ve.id_vendedor,
   ve.nombre
ORDER BY
   cantidad_ventas DESC,
   ve.nombre ASC;
```

Resultados obtenidos



vendedor	cantidad_ventas
Camila Ortiz	4
Diego Fuentes	3
Ignacio Vera	3
Javiera Soto	3
Paula Riquelme	3
Matías Herrera	2
Fernanda Lillo	0
Rubén Castro	0

Imagen 1: Conteo de ventas registradas por vendedor

Camila Ortiz registró 4 ventas. Diego Fuentes, Ignacio Vera, Javiera Soto y Paula Riquelme registraron 3 cada uno, y Matías Herrera registró 2. Fernanda Lillo y Rubén Castro figuran con 0. La suma de los conteos es 18, igual al total de registros de la tabla de ventas.

3.2. Ventas y unidades por tienda

Se calcularon el número de ventas registradas y el total de bicicletas vendidas por tienda. Cada registro de venta puede representar varias unidades.

Procedimiento

Desde tiendas se enlazó a vendedores y luego a ventas con dos LEFT JOIN. Al conservar las filas de la tabla inicial, el resultado incluye también las tiendas sin personal asignado. Se agrupó por identificador y nombre de tienda para obtener una fila por sucursal.

Se usó COUNT(v.id_venta) para contar las ventas y SUM(v.unidades) para sumar las bicicletas. En las tiendas sin ventas, COALESCE(SUM(v.unidades), 0) convierte la suma nula en cero. El orden descendente por unidades permite comparar el volumen de las sucursales.

```
SELECT
  t.nombre AS tienda,
  COUNT(v.id_venta) AS cantidad_ventas,
  COALESCE(SUM(v.unidades), 0) AS unidades_vendidas
FROM tiendas AS t
LEFT JOIN vendedores AS ve
  ON ve.id_tienda = t.id_tienda
LEFT JOIN ventas AS v
  ON v.id_vendedor = ve.id_vendedor
GROUP BY
  t.id_tienda,
  t.nombre
ORDER BY
  unidades_vendidas DESC,
  t.nombre ASC;
```

Resultados obtenidos

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tienda	cantidad_ventas	unidades_vendidas
Andes Concepción	6	14
Andes Providencia	7	11
Andes Maipú	5	10
Andes La Serena	0	0

Imagen 2: Ventas registradas y unidades vendidas por tienda

Concepción lideró en unidades con 14 en 6 ventas. Providencia registró la mayor cantidad de ventas, 7, y alcanzó 11 unidades. Maipú registró 5 ventas y 10 unidades, mientras que La Serena quedó con 0 en ambos indicadores. Los totales de las cuatro tiendas son 18 ventas y 35 unidades.

3.3. Tiendas sin vendedores asignados

El objetivo fue identificar las tiendas que no tienen vendedores asociados, mostrando además la comuna donde se ubican.

Procedimiento

La tabla tiendas se cruzó con vendedores mediante un LEFT JOIN por id_tienda. Cuando una tienda no tiene ninguna coincidencia, se conserva su información y las columnas del vendedor quedan en NULL.

El filtro WHERE ve.id_vendedor IS NULL selecciona esas filas. Se revisa el identificador del vendedor porque la condición que se quiere comprobar es la ausencia de una asignación de personal.

```
SELECT
  t.nombre AS tienda,
  t.comuna
FROM tiendas AS t
LEFT JOIN vendedores AS ve
  ON ve.id_tienda = t.id_tienda
WHERE ve.id_vendedor IS NULL
ORDER BY
  t.nombre ASC;
```

Resultados obtenidos



Imagen 3: Tienda sin vendedores asignados

La consulta devolvió una fila, Andes La Serena, ubicada en la comuna de La Serena. Como las ventas se asocian a las tiendas por medio de sus vendedores, la falta de personal asignado explica los ceros de esa sucursal en la consulta anterior.

3.4. Modelos sin ventas en junio de 2025

Se revisó el catálogo para identificar los modelos que no registraron ventas durante junio de 2025.

Procedimiento

La consulta parte de bicicletas y agrega al ON del LEFT JOIN el intervalo desde el 1 de junio inclusive hasta el 1 de julio exclusivo. Así, una venta fuera de junio no genera una coincidencia. Si un modelo carece de ventas dentro del período, permanece en la consulta con v.id_venta en NULL.

El filtro WHERE v.id_venta IS NULL selecciona los modelos que quedaron sin coincidencia. Si el filtro de fechas se trasladara al WHERE y se mantuviera IS NULL, el resultado quedaría vacío, porque las filas sin coincidencia también tienen la fecha en NULL.

```
SELECT
  b.modelo
FROM bicicletas AS b
LEFT JOIN ventas AS v
  ON v.id_bicicleta = b.id_bicicleta
  AND v.fecha >= '2025-06-01'
  AND v.fecha < '2025-07-01'
WHERE v.id_venta IS NULL
ORDER BY
  b.modelo ASC;
```

Resultados obtenidos

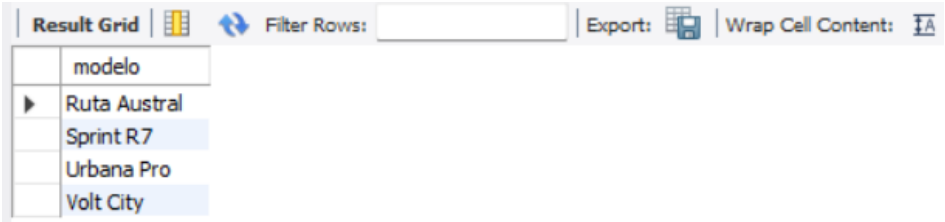


Imagen 4: Modelos sin ventas durante junio de 2025

Ruta Austral, Sprint R7, Urbana Pro y Volt City quedaron sin ventas registradas en junio. Los cuatro modelos representan la mitad del catálogo. La condición del mes permite concentrar el análisis en ese período incluso cuando un modelo sí se vendió en otra fecha.

3.5. Vendedores sin ventas y su tienda

Se buscó identificar a los vendedores sin ventas registradas e informar su tienda cuando existiera una asignación.

Procedimiento

Se usó vendedores como tabla inicial. El primer LEFT JOIN busca el nombre de la tienda en tiendas y conserva a los vendedores cuyo id_tienda es nulo. El segundo LEFT JOIN busca ventas mediante el identificador de cada vendedor.

La condición WHERE v.id_venta IS NULL deja solo a quienes carecen de registros en ventas. Al mantener la unión externa con tiendas, el resultado también incluye a quien no tiene sucursal asignada.

```
SELECT
  ve.nombre AS vendedor,
  t.nombre AS tienda
FROM vendedores AS ve
LEFT JOIN tiendas AS t
  ON t.id_tienda = ve.id_tienda
LEFT JOIN ventas AS v
  ON v.id_vendedor = ve.id_vendedor
WHERE v.id_venta IS NULL
ORDER BY
  ve.nombre ASC;
```

Resultados obtenidos

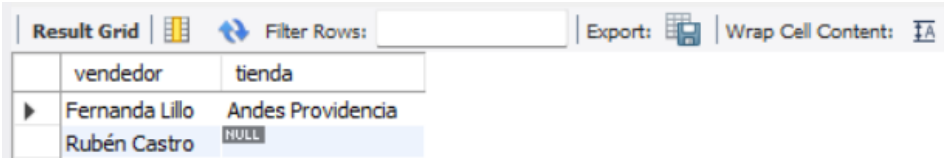


Imagen 5: Vendedores sin ventas y tienda asignada

Fernanda Lillo aparece asociada a Andes Providencia. Para Rubén Castro, la tienda aparece como NULL, lo que indica que su ficha no tiene una sucursal asignada. Los dos nombres coinciden con los vendedores cuyo conteo de ventas fue cero en la consulta 1.

3.6. Pares de vendedores de una misma tienda

Se formaron pares de vendedores asignados a una misma tienda, ubicando primero al que ingresó antes.

Procedimiento

La tabla vendedores se relacionó consigo misma con los alias v1 y v2. La condición de igualdad de id_tienda limita la comparación a personas de una misma sucursal. A continuación, se exige que la fecha de ingreso de v1 sea anterior a la de v2.

Si ambas fechas coinciden, el menor id_vendedor se ubica primero. Esta regla evita que una persona se empareje consigo misma o que se repita el par en orden inverso. La unión con tiendas incorpora el nombre de la sucursal a la salida.

```
SELECT
  v1.nombre AS vendedor_ingreso_anterior,
  v1.fecha_ingreso AS fecha_ingreso_anterior,
  v2.nombre AS vendedor_ingreso_posterior,
  v2.fecha_ingreso AS fecha_ingreso_posterior,
  t.nombre AS tienda
FROM vendedores AS v1
INNER JOIN vendedores AS v2
  ON v1.id_tienda = v2.id_tienda
  AND (v1.fecha_ingreso < v2.fecha_ingreso
    OR (v1.fecha_ingreso = v2.fecha_ingreso
      AND v1.id_vendedor < v2.id_vendedor))
INNER JOIN tiendas AS t
  ON t.id_tienda = v1.id_tienda
ORDER BY
  v1.fecha_ingreso ASC,
  v2.fecha_ingreso ASC,
  v1.nombre ASC,
  v2.nombre ASC;
```

Resultados obtenidos

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

vendedor_ingreso_anterior	fecha_ingreso_anterior	vendedor_ingreso_posterior	fecha_ingreso_posterior	tienda
Paula Riquelme	2022-11-02	Ignacio Vera	2024-08-19	Andes Concepción
Javiera Soto	2023-07-24	Matías Herrera	2024-01-15	Andes Maipú
Diego Fuentes	2024-01-15	Camila Ortiz	4-06-20	Andes Providencia
Diego Fuentes	2024-01-15	Fernanda Lillo	2025-02-03	Andes Providencia
Camila Ortiz	2024-06-20	Fernanda Lillo	2025-02-03	Andes Providencia

Imagen 6: Pares de vendedores ordenados por fecha de ingreso

Se obtuvieron cinco pares. Providencia generó tres a partir de sus tres vendedores. Maipú y Concepción generaron un par cada una, ya que cuentan con dos vendedores por tienda.

3.7. Tiendas con más de $2.000.000 en Montaña y Ruta

Se calculó cuánto vendió cada tienda en bicicletas de las categorías Montaña y Ruta y se seleccionaron las que superaron los $2.000.000.

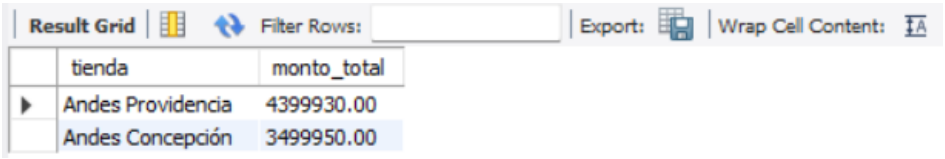
Procedimiento

Se enlazaron las tablas tiendas, vendedores, ventas y bicicletas para asociar cada venta con la tienda, la categoría y el precio del modelo. El WHERE filtra las dos categorías antes de calcular los totales.

El importe de cada registro se obtuvo multiplicando las unidades por el precio del modelo. La función SUM(v.unidades * b.precio) acumula esos importes por sucursal. El HAVING se aplica al total ya agrupado y exige que sea estrictamente superior a $2.000.000.

```
SELECT
   t.nombre AS tienda,
   SUM(v.unidades * b.precio) AS monto_total
FROM tiendas AS t
INNER JOIN vendedores AS ve
   ON ve.id_tienda = t.id_tienda
INNER JOIN ventas AS v
   ON v.id_vendedor = ve.id_vendedor
INNER JOIN bicicletas AS b
   ON b.id_bicicleta = v.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY
   t.id_tienda,
   t.nombre
HAVING SUM(v.unidades * b.precio) > 2000000
ORDER BY
   monto_total DESC;
```

Resultados obtenidos



tienda	monto_total
Andes Providencia	4399930.00
Andes Concepción	3499950.00

Imagen 7: Tiendas que superan $2.000.000 en Montaña y Ruta

Andes Providencia obtuvo $4.399.930, que se componen de $3.499.940 en Montaña y $899.990 en Ruta. Andes Concepción alcanzó $3.499.950. Estas fueron las dos sucursales que superaron el umbral.

3.8. Bicicletas con precio superior al promedio

Se identificaron los modelos cuyo precio es superior al promedio general del catálogo de Andes Bike.

Procedimiento

La subconsulta calcula AVG(precio) sobre los ocho modelos de bicicletas. Cada precio aporta una vez al promedio. La consulta principal compara el precio de cada modelo con ese valor mediante > y ordena los resultados de mayor a menor precio.

```
SELECT
   b.modelo,
   b.precio
FROM bicicletas AS b
WHERE b.precio > (
   SELECT AVG(precio)
   FROM bicicletas
)
ORDER BY
   b.precio DESC;
```

Resultados obtenidos



modelo	precio
Volt City	1249990.00
Sprint R7	1099990.00
Ruta Austral	899990.00
Trail Master	749990.00

Imagen 8: Modelos por encima del precio promedio del catálogo

Comprobación del promedio del catálogo

```
SELECT
   AVG(precio) AS precio_promedio_catalogo
FROM bicicletas;
```



precio_promedio_catalogo
657490.000000

Imagen 9: Promedio de los ocho precios del catálogo

Los ocho precios suman $5.259.920. Al dividir el total por ocho se obtiene el promedio de $657.490 que muestra la consulta auxiliar. Superaron ese valor Volt City ($1.249.990), Sprint R7 ($1.099.990), Ruta Austral ($899.990) y Trail Master ($749.990).

3.9. Vendedores sobre el promedio de unidades por venta

Se comparó la cantidad media de bicicletas por venta de cada vendedor con el promedio de unidades por venta de toda la base.

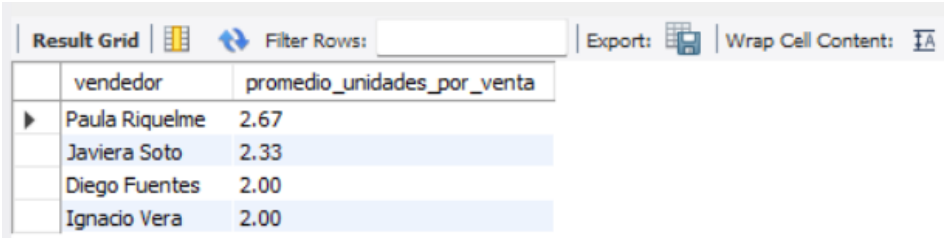
Procedimiento

Después de unir vendedores con ventas, se agruparon los registros por vendedor y se calculó AVG(v.unidades). La subconsulta calcula la referencia general sobre los 18 registros de venta. Al dividir 35 unidades por 18 ventas, el promedio general es de aproximadamente 1,94 unidades por venta.

El HAVING compara los promedios de cada vendedor con el valor general antes de redondearlos. La función ROUND muestra el resultado con dos decimales y el nombre se usa como segundo criterio de ordenamiento cuando hay un empate.

```
SELECT
  ve.nombre AS vendedor,
  ROUND(AVG(v.unidades), 2) AS promedio_unidades_por_venta
FROM vendedores AS ve
INNER JOIN ventas AS v
  ON v.id_vendedor = ve.id_vendedor
GROUP BY
  ve.id_vendedor,
  ve.nombre
HAVING AVG(v.unidades) > (
  SELECT AVG(unidades)
  FROM ventas
)
ORDER BY
  promedio_unidades_por_venta DESC,
  ve.nombre ASC;
```

Resultados obtenidos



vendedor	promedio_unidades_por_venta
Paula Riquelme	2.67
Javiera Soto	2.33
Diego Fuentes	2.00
Ignacio Vera	2.00

Imagen 10: Vendedores por encima del promedio de unidades por venta

Paula Riquelme obtuvo 2,67 unidades por venta y Javiera Soto, 2,33. Diego Fuentes e Ignacio Vera obtuvieron 2,00 cada uno. Los cuatro superaron el promedio general de 1,94 unidades por venta.

3.10. Monto total vendido por vendedor

Se obtuvo el monto total vendido por cada vendedor, incluyendo a quienes carecen de ventas registradas.

Procedimiento

La consulta parte de `vendedores` y utiliza dos `LEFT JOIN` para incorporar los registros de `ventas` y el precio desde `bicicletas`. Luego se agrupa por vendedor y se suman los importes de cada registro, calculados como unidades multiplicadas por precio.

Cuando no hay ventas asociadas, `SUM(v.unidades * b.precio)` devuelve `NULL` porque carece de importes para sumar. Para mostrar cero en esos casos se puede usar `COALESCE(SUM(v.unidades * b.precio), 0)`.

```
SELECT
  ve.nombre AS vendedor,
  SUM(v.unidades * b.precio) AS monto_total_vendido
FROM vendedores AS ve
LEFT JOIN ventas AS v
  ON v.id_vendedor = ve.id_vendedor
LEFT JOIN bicicletas AS b
  ON b.id_bicicleta = v.id_bicicleta
GROUP BY
  ve.id_vendedor,
  ve.nombre
ORDER BY
  monto_total_vendido DESC,
  ve.nombre ASC;
```

Resultados obtenidos

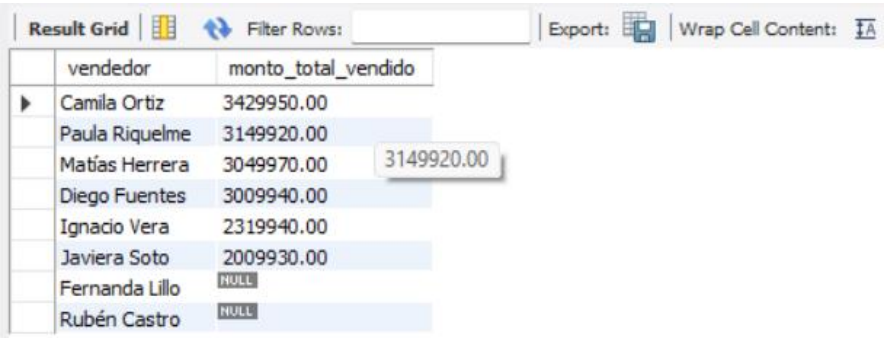


Imagen 11. Montos vendidos por vendedor en MySQL Workbench

Camila Ortiz obtuvo el mayor monto con $3.429.950 por cinco bicicletas distribuidas en cuatro ventas. Paula Riquelme le siguió con $3.149.920. Fernanda Lillo y Rubén Castro aparecen con `NULL`. La suma de los seis montos registrados es $16.969.650, que coincide con el total de la base.

4. Análisis de los resultados y conclusiones

4.1. Comparación de las sucursales

Al reunir los resultados por tienda, se observa que el liderazgo cambia según el indicador. La comparación incluye el monto vendido en Montaña y Ruta antes de aplicar el umbral de $2.000.000.

Sucursal	Vendedores asignados	Ventas	Unidades	Monto en Montaña y Ruta
Andes Providencia	3	7	11	$4.399.930
Andes Maipú	2	5	10	$1.299.980
Andes Concepción	2	6	14	$3.499.950
Andes La Serena	0	0	0	$0

Tabla 2. Comparación de personal asignado, ventas, unidades y monto por sucursal.

Providencia registró más ventas y un mayor monto en las dos categorías seleccionadas. Concepción concentró el 40 % de las unidades de la base y promedió 2,33 unidades por venta, frente a 1,57 en Providencia. Esta diferencia ayuda a interpretar por qué Concepción lidera en volumen con un registro de venta menos.

4.2. Seguimiento comercial

El cruce entre el precio del catálogo y el movimiento mensual mostró que Volt City, Sprint R7 y Ruta Austral superaban el precio promedio del catálogo y no registraron ventas en junio. Estos modelos pueden revisarse en el seguimiento comercial de ese mes junto con sus registros de otros períodos.

Fernanda Lillo figura en Providencia y Rubén Castro tiene su tienda sin asignar. Ambos carecen de ventas en la base. La revisión conjunta de las consultas 1, 3 y 5 permite distinguir la actividad comercial de la asignación de personal.

4.3. Conclusiones

La base relacional permitió cruzar la información comercial conservando las tiendas, los vendedores y los modelos que no tenían registros relacionados. Para lograrlo, fue necesario seleccionar la tabla inicial según la unidad de análisis y aplicar las condiciones de fecha en la unión cuando interesaba detectar modelos sin ventas en un período.

Las consultas también mostraron que el número de ventas, las unidades y el monto responden a preguntas distintas. Verificar los totales y revisar cómo se interpretan el cero y `NULL` permitió explicar los resultados de las capturas y relacionarlos con los datos de la carga oficial.